

Lecture 21: DNN Privacy---Model Inversion and Membership Inference

CS 580B1: Trustworthy Machine Learning

Fall 2024

Acknowledgments

Course slides derived from material created by Rajeev Alur and Osbert Bastani at UPenn ([link](#) to their course)

[Slides](#) by [Reza Shokri](#) on membership inference

Other relevant courses on Trustworthy Machine Learning:

[CS 329T](#) at Stanford

[CS 521](#) at UIUC

[ECE1784H/CSC2559H](#) at U Toronto

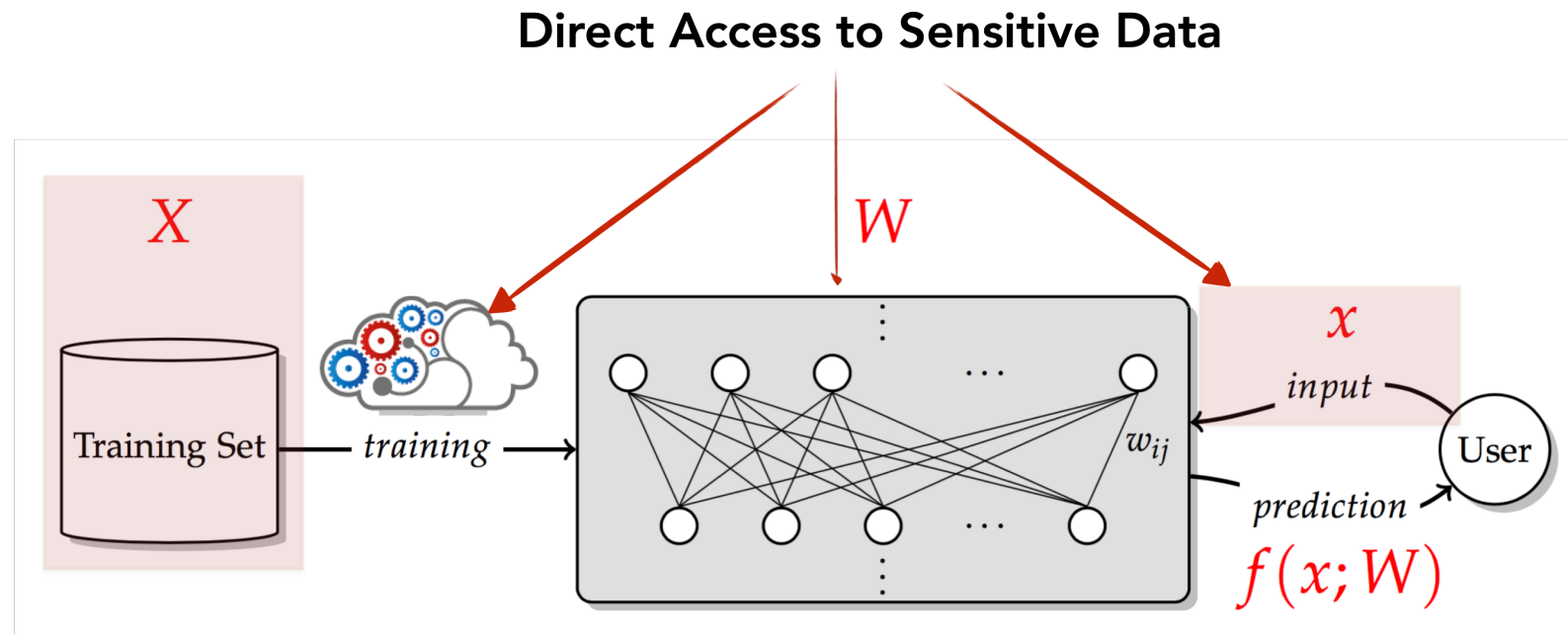
Course logistics

- Reviews:
 - Review due tomorrow at 11:59 PM
 - Paper to review: [Stealing Part of a Production Language Model](#) by Carlini et al.

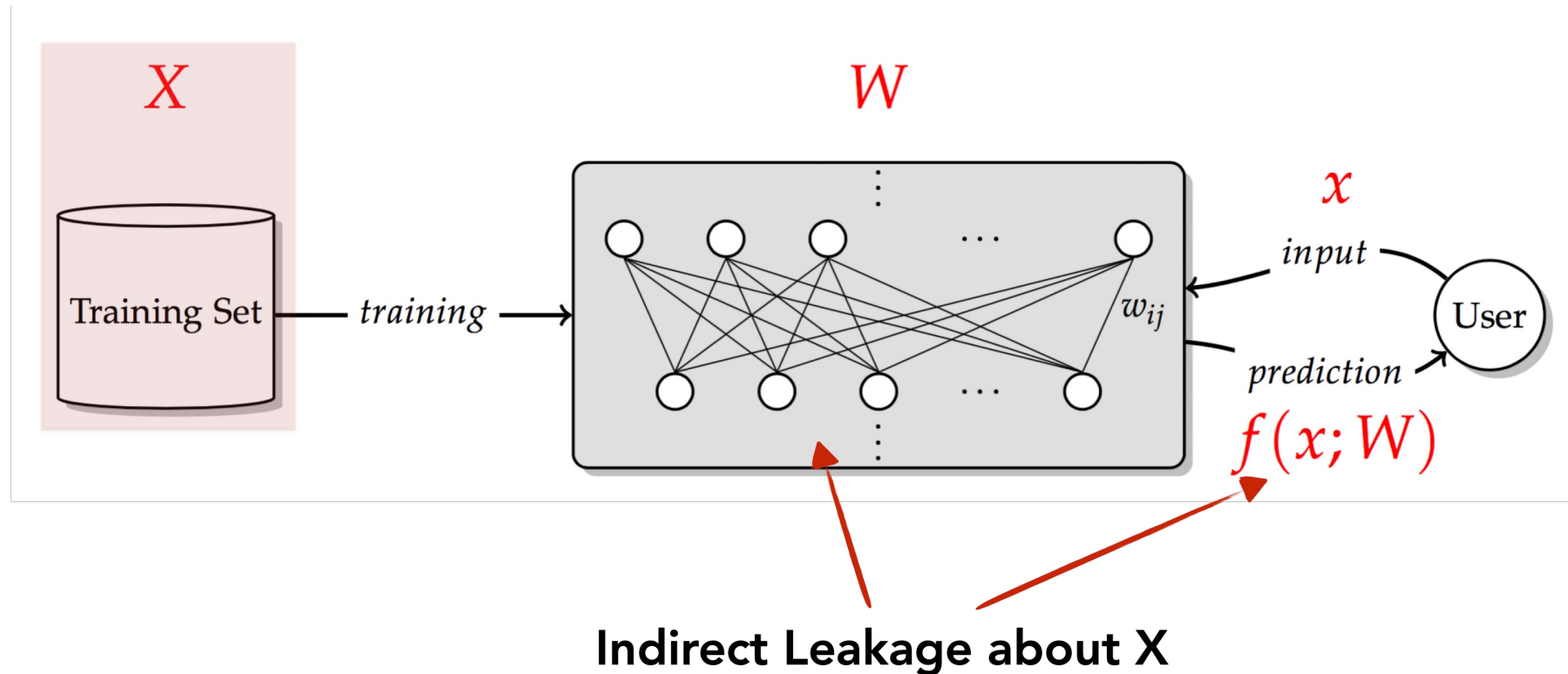
Recap: What is *trustworthy* machine learning?

- Techniques for evaluating *trustworthiness* of ML models and learning *trustworthy* models
- Metrics of *trustworthiness*:
 - Accuracy
 - Robustness
 - Interpretability
 - Alignment with human values
 - Privacy
 - Being well-calibrated
 - Fairness
 - ...

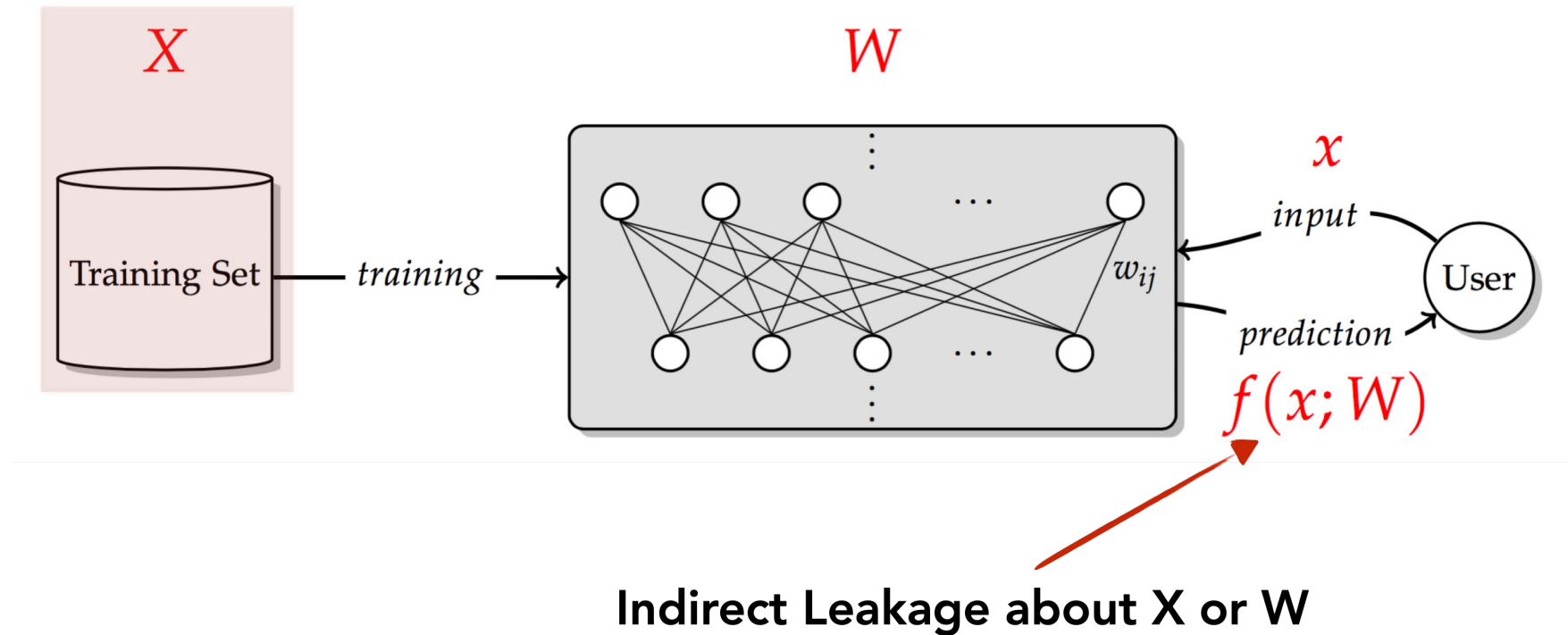
Direct privacy risks



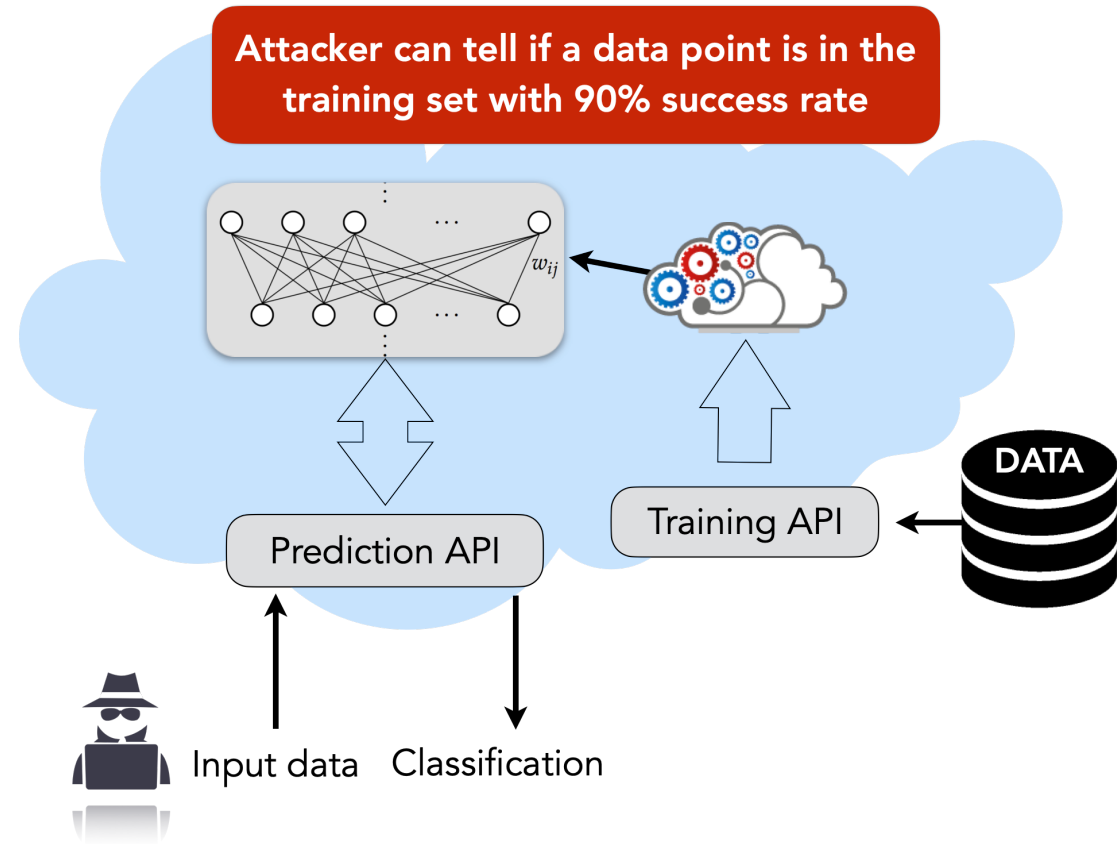
Indirect privacy risks



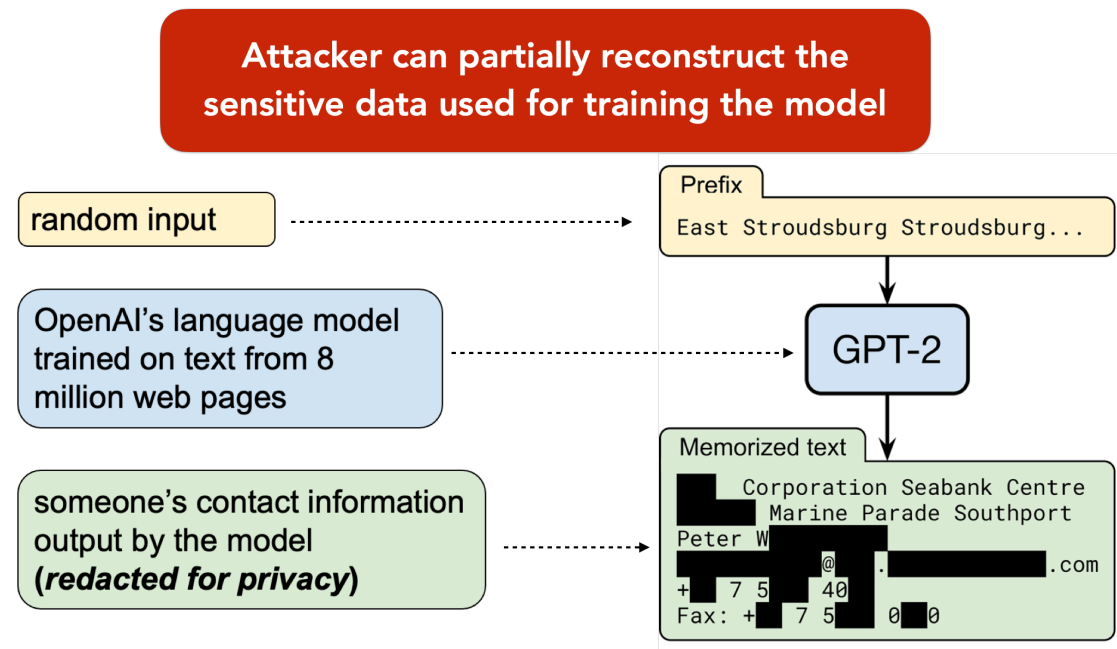
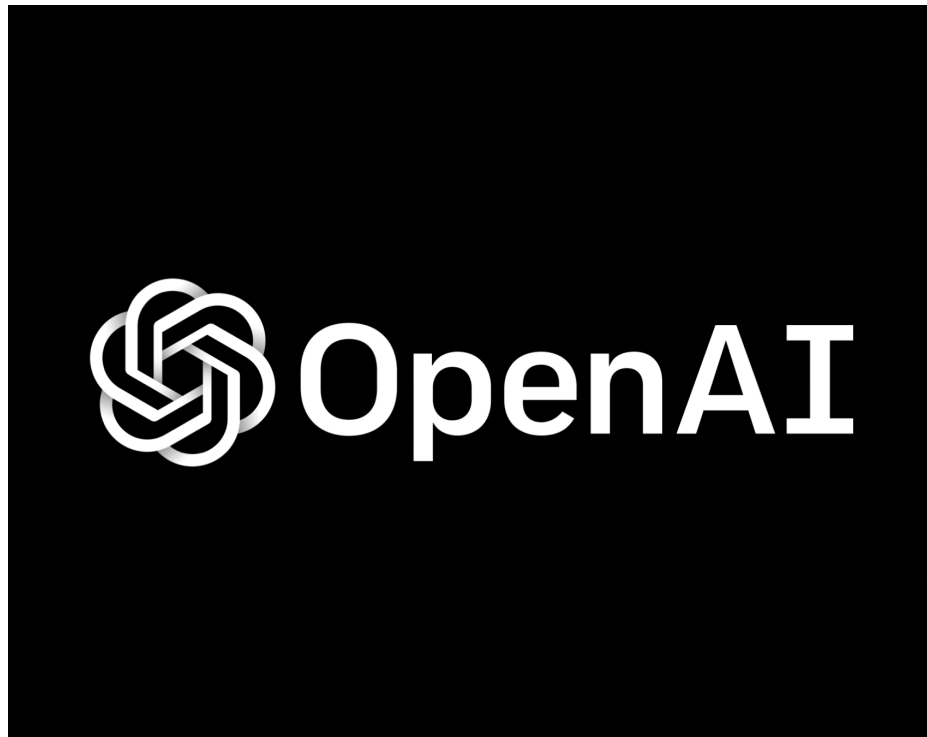
Indirect privacy risks



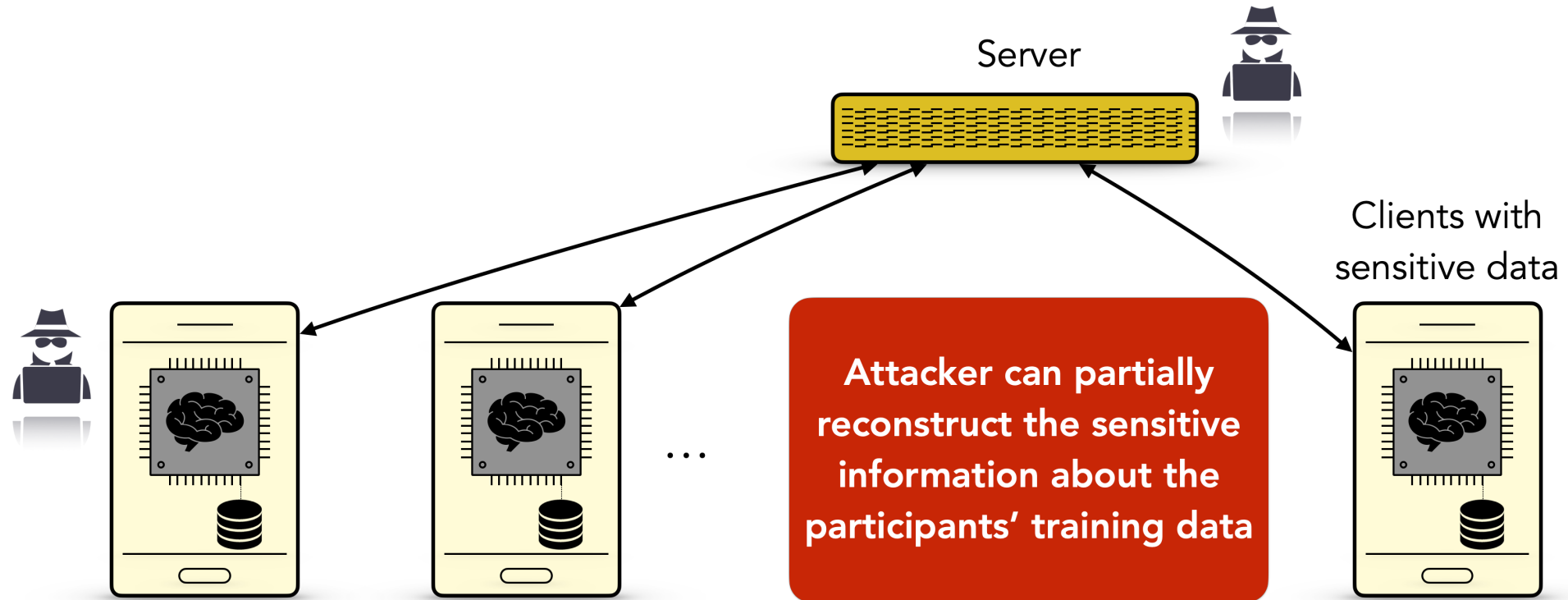
Privacy attacks against Machine Learning as a Service platforms



Privacy attacks against LLMs



Privacy attacks against Federated Learning algorithms



Shokri, Auditing Data Privacy in Machine Learning: A Comprehensive Introduction, CCS'22 Tutorial

Nasr et al., Comprehensive Privacy Analysis of Deep Learning: Passive and Active White-box Inference Attacks against Centralized and Federated Learning, SP'19

Melis et al., Exploiting Unintended Feature Leakage in Collaborative Learning, SP'19

Zhang et al., Leakage of Dataset Properties in Multi-Party Machine Learning, Usenix Security'21

Today's lecture

Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures by Fredrikson et al., CCS'15

Membership Inference Attacks against Machine Learning Models by Shokri et al., SP'17

[Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures](#) by Fredrikson et al.

Model inversion attack



Figure 1: An image recovered using a new model inversion attack (left) and a training set image of the victim (right). The attacker is given only the person's name and access to a facial recognition system that returns a class confidence score.

Model inversion: Threat model

- White-box access to the model---architecture and weights **W**
- No information available about the training data **X**
- Given predicted label y , can adversary reconstruct the input x ?

Attack algorithm

```
 $x_0 \leftarrow \text{Initialize}()$   
 $t \leftarrow 0$   
Until budget exhaustion:  
     $x_{t+1} \leftarrow x_t - \lambda \nabla_x (L(f(x_t), y))$   
     $t \leftarrow t + 1$   
return ( $x_t$ )
```

- Start with some random input vector
- Use gradient descent in input space to maximize model's confidence in the target prediction
- Black-box variant:
 - Assume API access to the model such that the model returns the prediction along with the confidence
 - Estimate gradient at x_t via multiple calls to the API

Empirical results

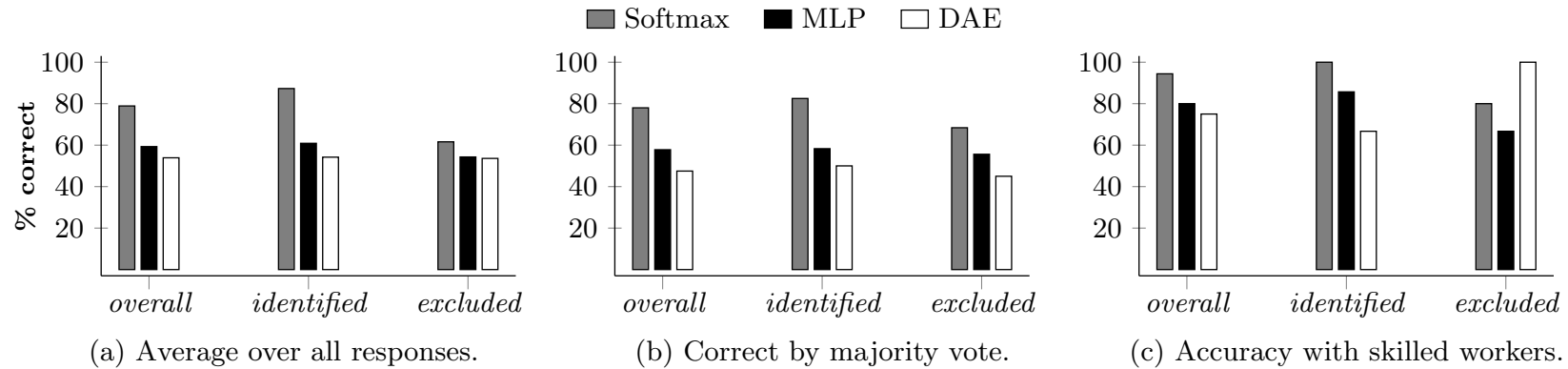


Figure 9: Reconstruction attack results from Mechanical Turk surveys. “Skilled workers” are those who completed at least five MTurk tasks, achieving at least 75% accuracy.

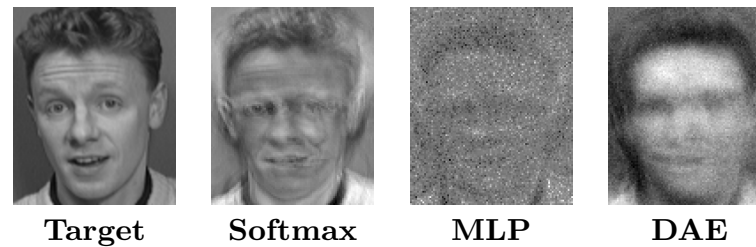
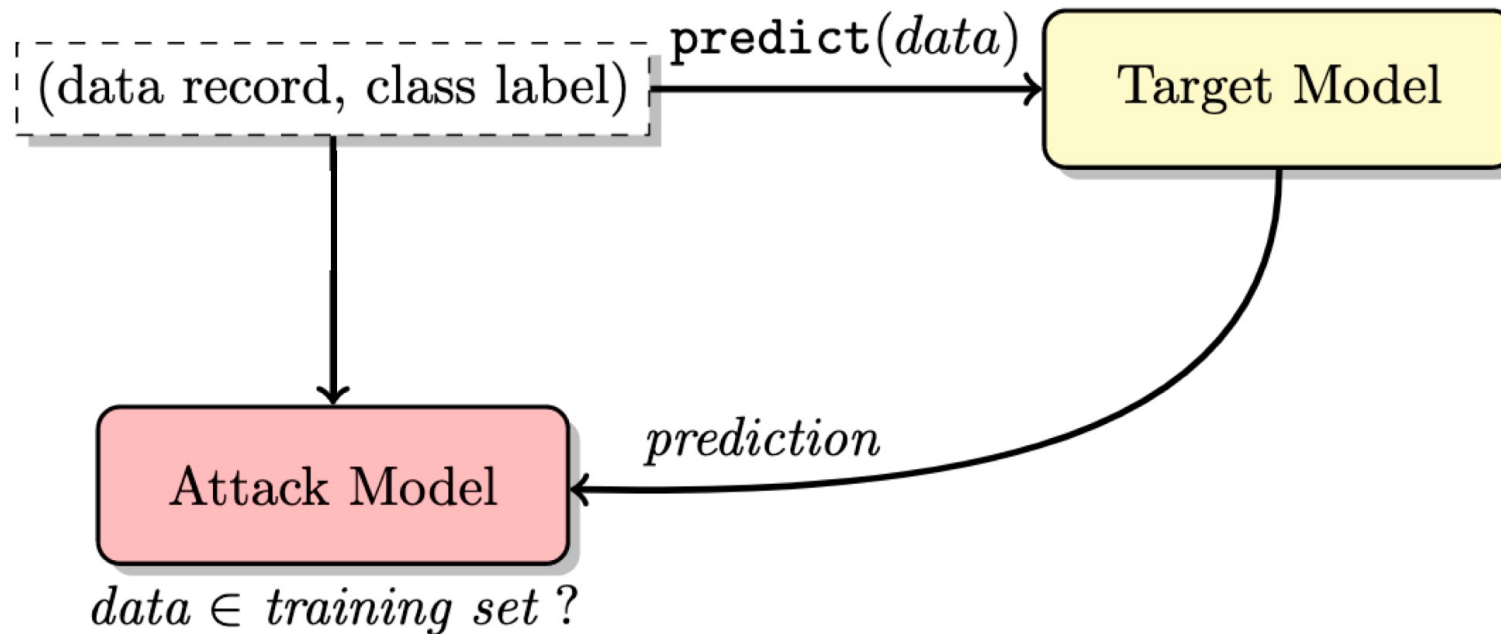


Figure 10: Reconstruction of the individual on the left by Softmax, MLP, and DAE.

Membership Inference Attacks against Machine Learning Models by Shokri et al.

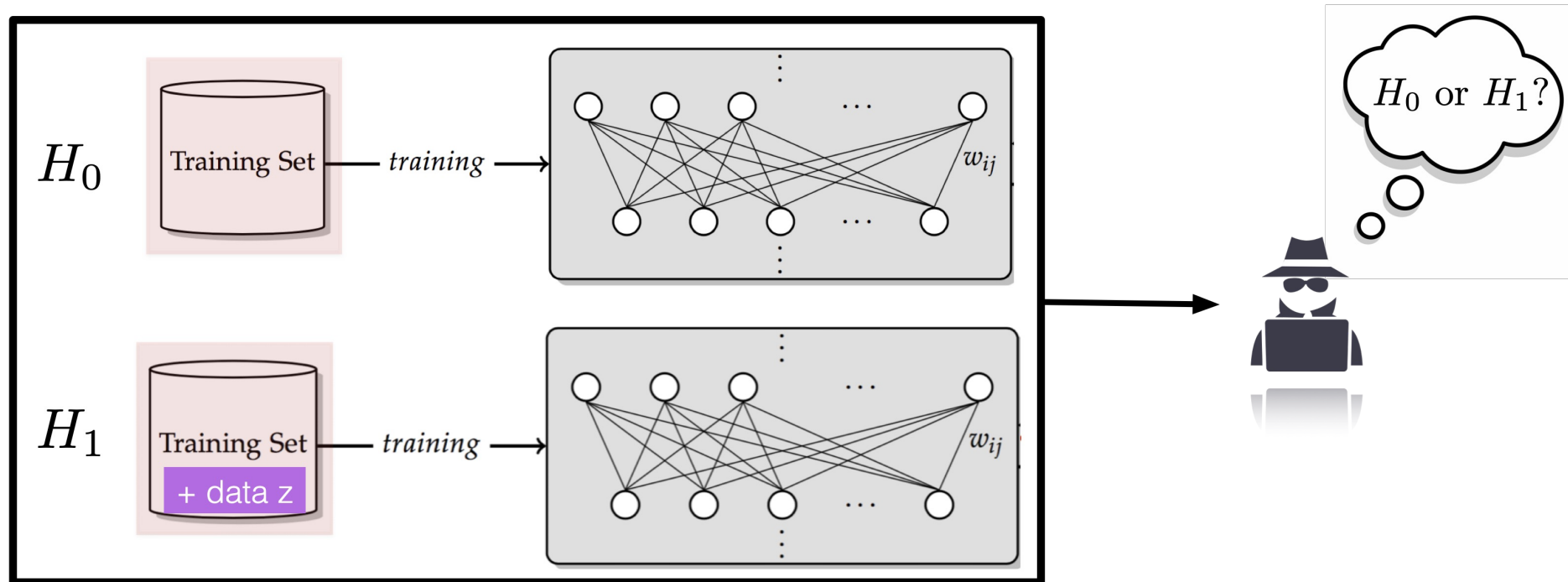
Membership inference attack

Given a model, can an adversary infer whether a particular data point is part of its training set?



Membership Inference: Threat model

- ML-as-a-service
- User can query the ML model via APIs to make predictions
- API returns prediction along with the confidence score
(Recall the ML classifiers generate a score for every label)
- Can an adversary infer whether a particular data point is part of model's training set?



Membership inference attack: Basic idea

Step 1: Synthesize a dataset that is (distributionally) similar to the training dataset used for the *target model*

Step 2: Using the synthesized dataset, train *shadow model(s)* that are similar in behavior to the *target model*

Step 3: Use *shadow models* to create a dataset with elements of the form (prediction, class label, “in” / “out”)

Step 4: Train *attack model* that can predict “in” / “out” based on prediction vector and true class label

Step 1: Synthesize dataset

Intuition: Records that are classified by the *target model* with high confidence should be statistically similar to the target's training dataset

Approach: Use local search to find records (inputs) that are classified by the *target model* with a high confidence

- Start with a random input
- Randomly perturb the input and check if it classified with a high confidence
- Repeat

Step 2: Train *shadow models*

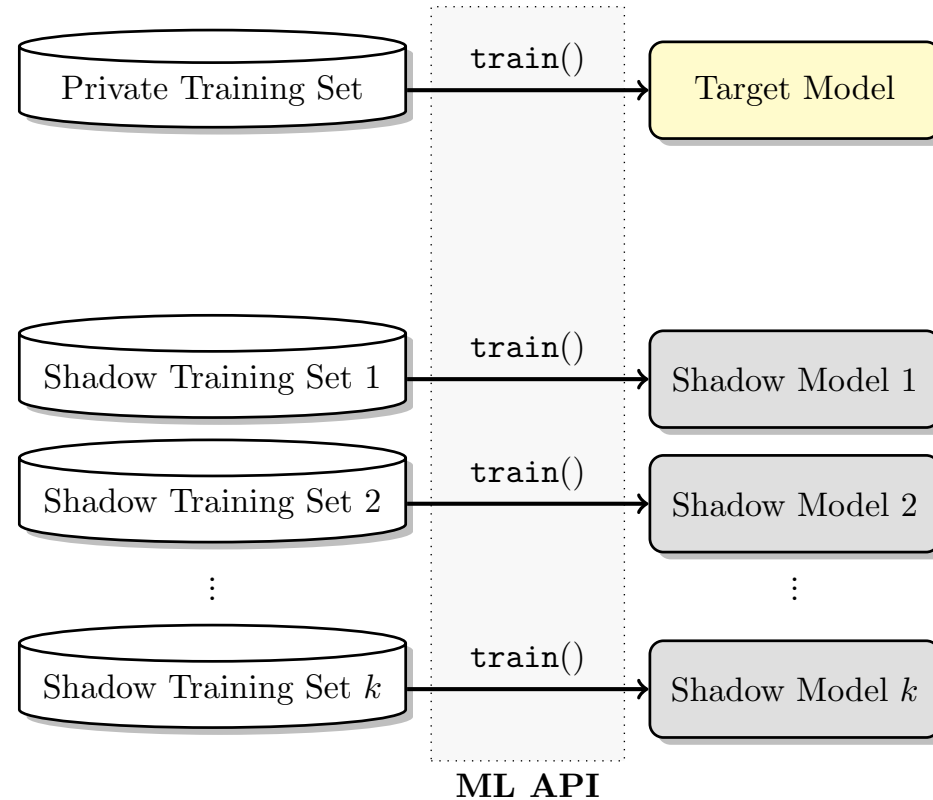


Fig. 2: Training shadow models using the same machine learning platform as was used to train the target model. The training datasets of the target and shadow models have the same format but are disjoint. The training datasets of the shadow models may overlap. All models' internal parameters are trained independently.

Step 3: Create dataset using *shadow models*

Step 4: Train *attack model*

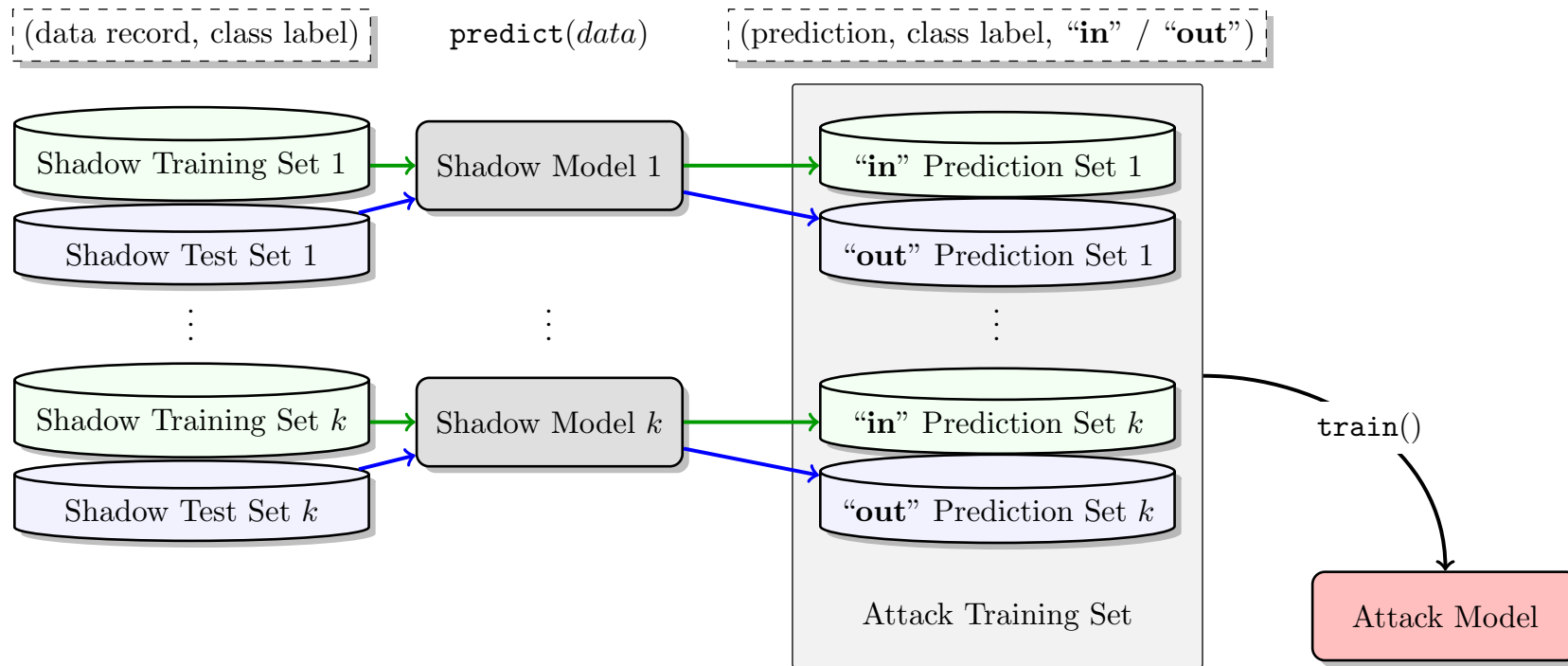


Fig. 3: Training the attack model on the inputs and outputs of the shadow models. For all records in the training dataset of a shadow model, we query the model and obtain the output. These output vectors are labeled “in” and added to the attack model’s training dataset. We also query the shadow model with a test dataset disjoint from its training dataset. The outputs on this set are labeled “out” and also added to the attack model’s training dataset. Having constructed a dataset that reflects the black-box behavior of the shadow models on their training and test datasets, we train a collection of c_{target} attack models, one per each output class of the target model.

Empirical evaluation

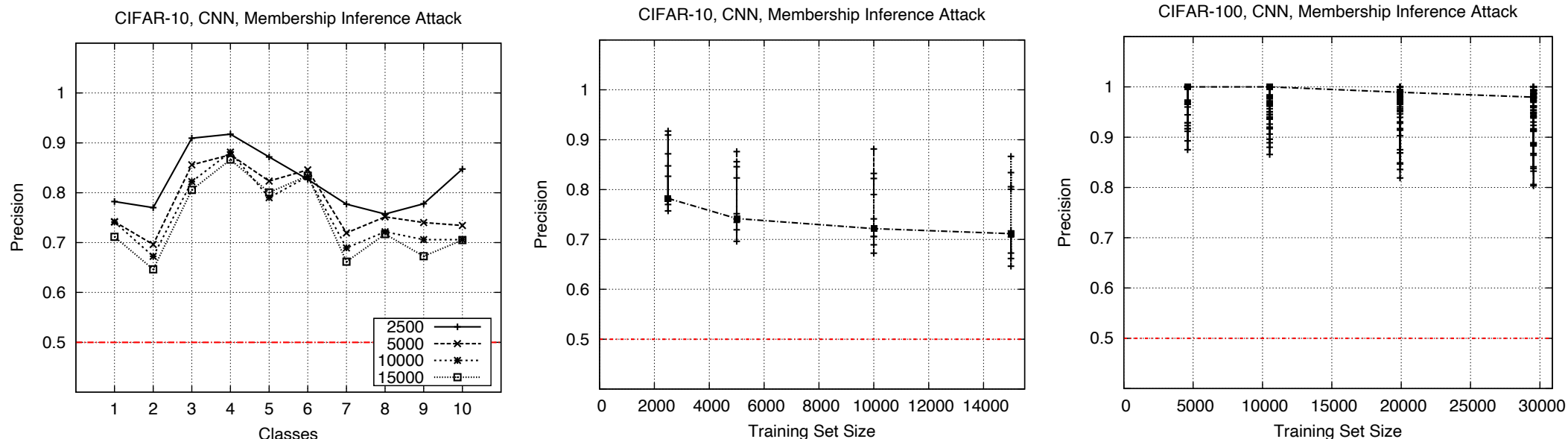
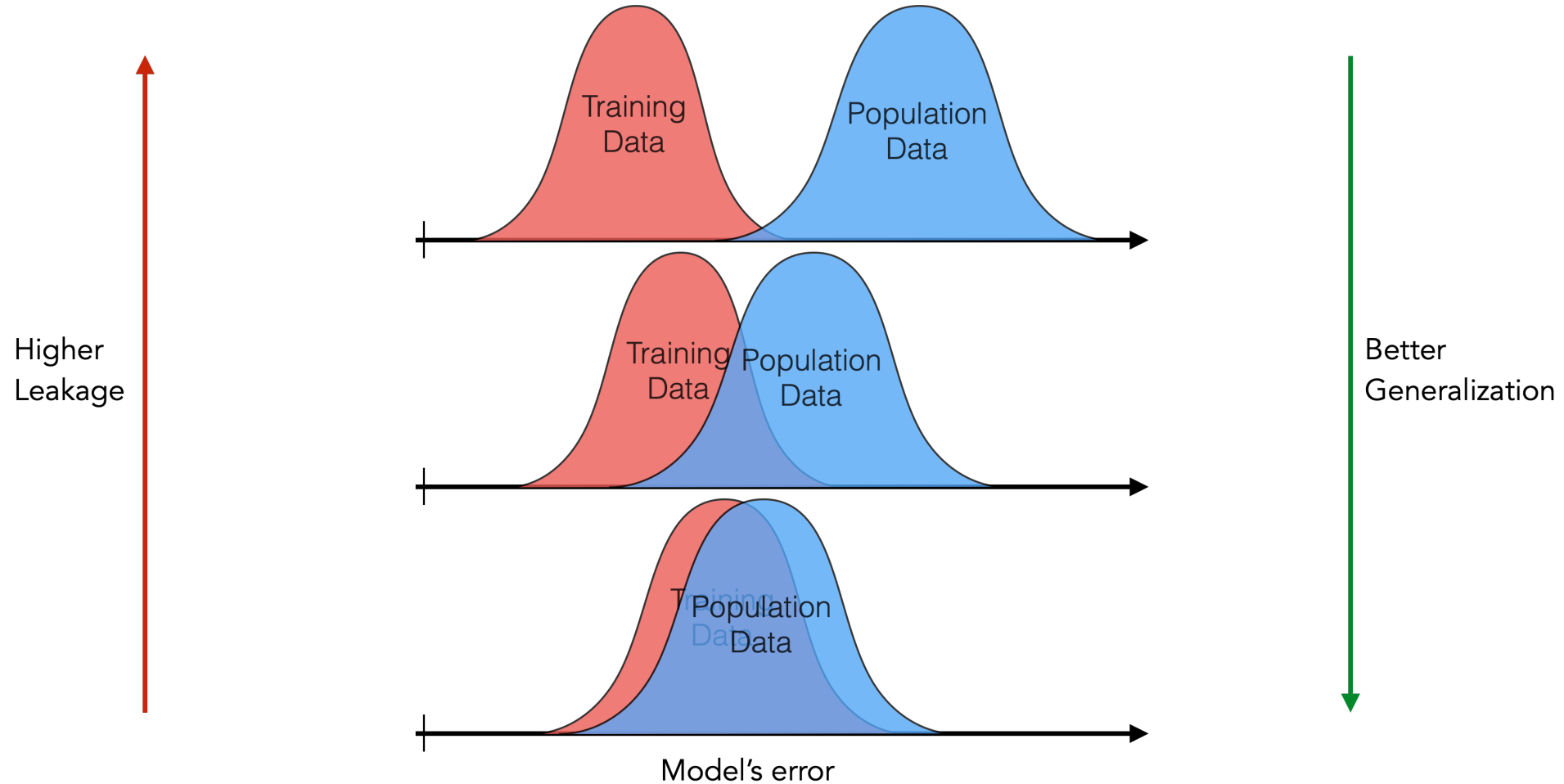
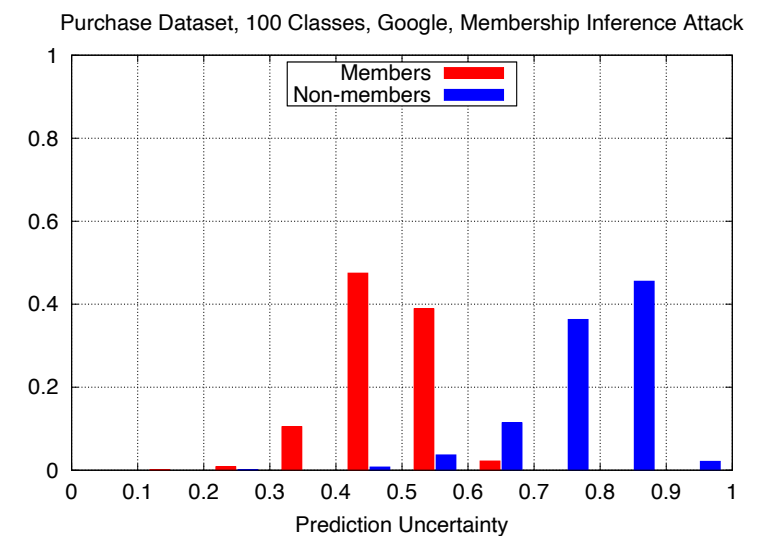
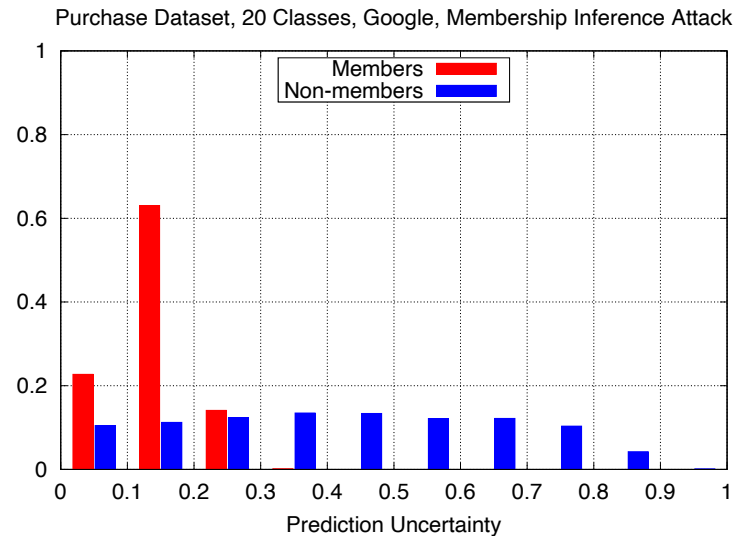
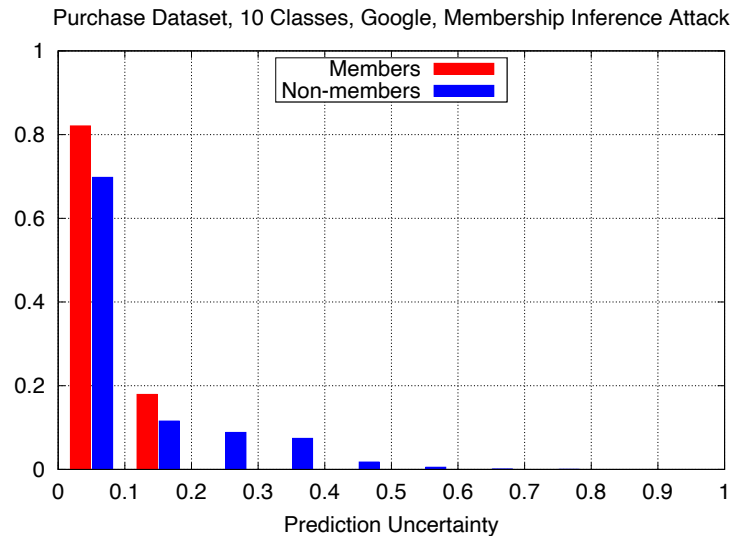


Fig. 4: Precision of the membership inference attack against neural networks trained on CIFAR datasets. The graphs show precision for different classes while varying the size of the training datasets. The median values are connected across different training set sizes. The median precision (from the smallest dataset size to largest) is 0.78, 0.74, 0.72, 0.71 for CIFAR-10 and 1, 1, 0.98, 0.97 for CIFAR-100. Recall is almost 1 for both datasets. The figure on the left shows the per-class precision (for CIFAR-10). Random guessing accuracy is 0.5.

Why does the membership inference attack work?



Why does the membership inference attack work?



where prediction uncertainty is normalized entropy of model's prediction vector $:= \frac{-1}{\log(n)} \sum_i p_i \log(p_i)$